Taller 2

Taller 2

PCA

1. Considerar la base `df_pca_B.csv`

a. Ajustar el algoritmo PCA a la base:

```
df_pca=_______
pca_obj=PCA(n_components=_____)
pca_obj.fit(df_pca)
```

b. Elegir un número de componentes principales adecuados.

```
def diagPCA(pca_obj):
  var_exp = pca_obj._____
  cum_var = np._____(var_exp)
  summary_df = pd.DataFrame({'Standard deviation': np.sqrt(pca_obj._______),
      'Proportion of Variance': var_exp,'Cumulative Proportion': cum_var},
      index=[f"PC{i+1}" for i in range(len(var_exp))])
  print("Summary:")
  print(summary_df)
  fig, ax = plt.subplots()
  ax.plot(range(1, len(_____) + 1), var_exp, 'o-', color='blue')
  ax.set_xlabel("Componente Principal")
  ax.set_ylabel("Proporción de Varianza Explicada")
  ax.set_title("Screeplot")
  ax.grid(True, linestyle='--', alpha=0.5)
  plt.show()
```

b. Reconstruir la base reduciendo la dimensión a dicho número utilizando los resultados de svd y los resultados de pca.

```
def recDataRed_pca(df,pca_obj,k,ms,sds):
  scores = pca_obj._______(df)[_______]
  loadings = pca_obj._______[_______]

  X_rec_sc = np.dot(scores, loadings)
  X_rec = (X_rec_sc * _____) + ____
  df_rec=pd.DataFrame(X_rec, index=df.index, columns=df.columns)
  return df_rec

def recDataRed_svd(df,k,ms,sds):

  U, s, Vt = np.linalg.svd(df, full_matrices=False)

  U_k = U[_______]
  S_k = np.diag(s[_____])
  Vt_k = Vt[____]
```

```
  X_rec_sc = np.dot(U_k,np.dot(S_k,Vt_k))
  X_rec = (X_rec_sc * ____) + ____
  df_rec=pd.DataFrame(X_rec, index=df.index, columns=df.columns)
  return df_rec
```

c. Graficar los datos proyectados en los componentes principales elegidos. Agregar color según alguna otra variable y/o cuán bien representadas están las observaciones en dicha proyección

```
def calcular_cos2_obs(scores):
    scores_sq = scores**2
    dist2_total = scores_sq.sum(axis=1)
    cos2_df = scores_sq.div(dist2_total, axis=0)
    return cos2_df

def scorePlot_PCA(df,pca_obj, axes=[1, 2], col_ch=None, tit=""):
  pc_x, pc_y = f"PC{axes[0]}", f"PC{axes[1]}"
  nComp=pca_obj._____.shape[0]
  scores=pca_obj._____(df)
  scores=pd.DataFrame(scores,index=df.index,columns=["PC"+str(i+1) for i in range(nComp)])
  fig, ax = plt.subplots()

  if col_ch is None:
    ax.scatter(____[pc_x], ____[pc_y], alpha=0.7)
  else:
    if col_ch in df.columns:
      scatter = ax.scatter(____[pc_x], ____[pc_y], c=df[____], cmap="viridis", alpha=0.7)
      plt.colorbar(scatter, ax=ax, label=col_ch)
    elif col_ch in scores.columns:
      scatter = ax.scatter(____[pc_x], _____[pc_y], c=scores[____], cmap="viridis", alpha=0.7)
      plt.colorbar(scatter, ax=ax, label=col_ch)
    elif col_ch=="cos2":
      cos2_df = calcular_cos2_obs(scores)
      cos2_plano = cos2_df[pc_x] + cos2_df[pc_y]
      scatter = ax.scatter(____[pc_x], ____[pc_y], c=cos2_plano, cmap="viridis", alpha=0.7)
      cmap = mpl.colormaps['coolwarm']
      norm = mcolors.Normalize(vmin=0, vmax=1)
      sm = cm.ScalarMappable(cmap=cmap, norm=norm)
      sm.set_array([])
      cbar = plt.colorbar(sm, ax=ax, fraction=0.05, pad=0.04)
      cbar.set_label("cos2")
    else:
      raise ValueError("Opción de color inválida")

  ax.set_xlabel(pc_x)
  ax.set_ylabel(pc_y)
  ax.set_title(tit)
  ax.grid(True, linestyle='--', alpha=0.5)
  return ax
```

d. Graficar las correlaciones entre variables y componentes principales e identificar qué variable se correlaciona con cada componente (identificar el signo también).

```
def corPlot_PCA(df,pca_obj, axes=[1, 2], tit="", ax=None):
  pc_x, pc_y = f"PC{axes[0]}", f"PC{axes[1]}"
  nComp=pca_obj._____.shape[0]
```

```
  scores=pca_obj._____(df)
  scores=pd.DataFrame(scores,index=df.index,columns=["PC"+str(i+1) for i in range(nComp)])
  corrs=pd.DataFrame({pc_x:[___[var].corr(____[pc_x]) for var in df.____],
  pc_y:[___[var].corr(____[pc_y]) for var in df._____]},index=df._____)
  corrs["cos2"]=corrs[pc_x]**2+corrs[pc_y]**2
  cmap = mpl.colormaps['coolwarm']
  norm = mcolors.Normalize(vmin=0, vmax=1)

  fig, ax = plt.subplots()
  for i in range(corrs.shape[0]):
    color_var = cmap(norm(corrs.loc[corrs._____[i],"cos2"]))
    ax.annotate(corrs._____[i], (1.1*corrs.loc[corrs._____[i],pc_x], 1.1*corrs.loc[corrs._____[i],pc_y])
    ax.arrow(0, 0, corrs.loc[corrs._____[i],pc_x], corrs.loc[corrs._____[i],pc_y], color=color_var, alpl

  sm = cm.ScalarMappable(cmap=cmap, norm=norm)
  sm.set_array([])
  cbar = plt.colorbar(sm, ax=ax, fraction=0.05, pad=0.04)
  cbar.set_label("cos2")
  circulo = plt.Circle((_____, _____), _____, color='gray', fill=False, alpha=0.5)
  ax.add_patch(circulo)
  ax.set_xlim(-1.2, 1.2)
  ax.set_ylim(-1.2, 1.2)
  ax.set_aspect('equal')
  ax.set_xlabel(pc_x)
  ax.set_ylabel(pc_y)
  ax.set_title(tit)
  ax.grid(True, linestyle='--', alpha=0.5)
  return ax
```

e. Realizar un biplot que permite ver tanto representación de observaciones como de variables en simultáneo. Agregar color según alguna variable de la base original que permita detectar un patrón.

```
def biPlot_PCA(df,pca_obj, axes=[1, 2], tit="",col_ch=None):
  pc_x, pc_y = f"PC{axes[0]}", f"PC{axes[1]}"
  scores=pca_obj._____(df)
  scores=pd.DataFrame(scores,index=df.index,columns=["PC"+str(i+1) for i in range(len(df.columns.values)
  covs=pd.DataFrame({pc_x:[df[____].cov(scores[___]) for var in ___.columns],pc_y:[df[___].cov(scores[_

  cmap = mpl.colormaps['coolwarm']
  norm = mcolors.Normalize(vmin=0, vmax=1)

  fig, ax = plt.subplots()
  if col_ch is None:
    ax.scatter(___[pc_x], ___[pc_y], alpha=0.4, color='black')
  else:
    if col_ch in df.columns:
      scatter = ax.scatter(___[pc_x], ___[pc_y], c=___[___], cmap="viridis", alpha=0.4)
      plt.colorbar(scatter, ax=ax, label=____)
    else:
      raise ValueError("Opción de color inválida")


  for i in range(_____):
    ax.annotate(covs.____[i], (1.1*covs.loc[covs._____[i],____], 1.1*covs.loc[covs.____[i],____]), colo
```

```
    ax.arrow(0, 0, covs.loc[covs.____[i],____], covs.loc[covs.____[i],____], color="red", alpha=0.3, hea

  ax.set_xlabel(pc_x)
  ax.set_ylabel(pc_y)
  ax.set_title(tit)
  ax.grid(True, linestyle='--', alpha=0.5)
  return ax
```

Análisis Factorial

2. Considerar la base `df_af_B.csv`:

a. Elegir un número de factores adecuado.

```
df_af=____
cors=df_af.____()
eigs=np.linalg.____(cors).____
print(eigs)
anfact=FactorAnalysis(n_components=____)
anfact.fit(df_af)
loadings = pd.DataFrame(anfact.____.T,index=df_af.columns,columns=[f"Factor{i+1}" for i in range(anfact
print(loadings)
uniquenesses = pd.Series(anfact.____,index=df_af.____)
print(uniquenesses)
```

b. Representar las correlaciones entre variables y los factores. Identificar las variables correlacionadas con cada factor, considerando el signo.

```
def RepFact(df,anfact_obj, facts=[1, 2]):
  nFacts=anfact_obj.____.shape[____]
  loadings = pd.DataFrame(anfact_obj.____.T,index=df.columns,columns=[f"Factor{i+1}" for i in range(nFa
  fig, ax = plt.subplots()
  f1, f2 = f"Factor{facts[0]}", f"Factor{facts[1]}"

  ax.scatter(____[f1], ____[f2], color='blue')
  for i in range(loadings.shape[0]):
    ax.annotate(loadings.____[i], (loadings.loc[loadings.____[i],f1], loadings.loc[loadings.____[i],f2]

  ax.set_xlim(-1.2, 1.2)
  ax.set_ylim(-1.2, 1.2)
  ax.set_aspect('equal')
  ax.set_xlabel(f1)
  ax.set_ylabel(f2)
  ax.set_title("Cargas Factoriales")
  ax.grid(True, linestyle='--', alpha=0.5)
  return ax
```

c. Representar las observaciones proyectadas sobre los factores. Agregar color por factores restantes, si es que los hubiere y agregar color por alguna variable de la base original que considere pertinente.

```
def RepScore(df,anfact_obj, facts=[1, 2], col_ch=None):
    nFacts=anfact_obj.____.shape[____]
    scores = pd.DataFrame(anfact_obj.____(df),columns=[f"Factor{i+1}" for i in range(nFacts)],
    index=df.____)
    fig, ax = plt.subplots()
    f1, f2 = f"Factor{facts[0]}", f"Factor{facts[1]}"
```

```
    if col_ch is None:
        ax.scatter(scores[f1], scores[f2], alpha=0.5)
    else:
      if col_ch in scores.columns:
        scatter = ax.scatter(____[f1], ____[f2], c=____[____], cmap="viridis", alpha=0.5)
        plt.colorbar(scatter, ax=ax, label=____)
      elif col_ch in df.columns:
        scatter = ax.scatter(____[f1], ____[f2], c=____[____], cmap="viridis", alpha=0.5)
        plt.colorbar(scatter, ax=ax, label=____)
      else:
        raise ValueError("Opción de color inválida")

    ax.set_aspect('equal')
    ax.set_xlabel(f1)
    ax.set_ylabel(f2)
    ax.set_title("Scores Factoriales")
    ax.grid(True, linestyle='--', alpha=0.5)
    return ax
```

CA

3. Considerar la tabla de contingencia `Tab_ca_B.csv`:

```
Tab_ca=____
Tab_ca.index=[f"A{i+1}" for i in range(Tab_ca.shape[____])]
Tab_ca.columns=[f"B{i+1}" for i in range(Tab_ca.shape[____])]
```

a. Calcular la matriz de valores esperados según independencia.

```
def EMat(Tab):
  matTab=Tab.values
  nR, nC = matTab.____
  sR = matTab.sum(axis=____).reshape(-1, 1)
  sC = matTab.sum(axis=____).reshape(1, -1)
  sT = matTab.sum()

  E = np.dot(____,____) / ____
  E= pd.DataFrame(E, index=Tab.____, columns=Tab.____)
  return E
```

b. Calcular la matriz de desviaciones entre las observaciones y los valores esperados.

```
def CMat(Tab, E):
    C = (____.values - ____.values) / np.sqrt(____.values)
    C = pd.DataFrame(C, index=Tab.____, columns=Tab.____)
    return C
```

c. Elegir un número de ejes adecuado según la varianza acumulada.

```
def pVarEx(C):
    U, s, Vt = np.linalg.svd(____.values)
    autovalores = s**____
    cum_perc = np.cumsum(____) * 100 / np.sum(____)
    pVar=pd.DataFrame({'Autovalores': autovalores, 'Porcentaje_Acumulado': cum_perc})
    return pVar
```

d. Representar cada fila y cada columna en el eje correspondiente e identificar combinaciones de filas frecuentes e infrecuentes.

```
def rScore(C, Tab, k):
  nRs=Tab.values.sum(axis=____)
  C_mat = C.values
  CCt = np.dot(____,____)
  eig_vals, eig_vecs = np.linalg.____(CCt)
  eig_vals=eig_vals.real[____]
  eig_vecs=eig_vecs.real[____]

  A_aux = np.diag(1.0 / np.sqrt(____))
  rSc = np.dot(np.dot(A_aux, ____),np.diag(np.sqrt(____)))

  df_rSc = pd.DataFrame(rSc, columns=[f"Eje{i+1}" for i in range(k)])
  df_rSc['Lab'] = Tab.____
  return df_rSc

def cScore(C, Tab, k):
    nCs=Tab.values.sum(axis=____)
    C_mat = C.values
    CtC = np.dot(____,____)
    eig_vals, eig_vecs = np.linalg.____(CtC)

    eig_vals = eig_vals.real[____]
    eig_vecs = eig_vecs.real[____]

    B_aux = np.diag(1.0 / np.sqrt(____))
    cSc = np.dot(B_aux,np.dot(____,np.diag(np.sqrt(____))))

    df_cSc = pd.DataFrame(cSc, columns=[f"Eje{i+1}" for i in range(k)])
    df_cSc['Lab'] = C.____
    return df_cSc

def RepGraf(rS, cS, ejs=[1, 2]):
    fig, ax = plt.subplots()
    e1, e2 = f"Eje{ejs[0]}", f"Eje{ejs[1]}"

    ax.scatter(rS[e1], rS[e2], color="w", alpha=0,s=10)

    for i in range(rS.shape[____]):
      ax.text(rS.loc[rS.____[i],____], rS.loc[rS.____[i],____],s= rS.loc[rS.____[i],____], color="blue"
    for i in range(cS.shape[____]):
      ax.text(cS.loc[cS.____[i],____], cS.loc[cS.____[i],____],s= cS.loc[cS.____[i],____], color="red")

    ax.set_xlabel(e1)
    ax.set_ylabel(e2)
    ax.set_title("Análisis de Correspondencias")
    ax.grid(True, linestyle='--', alpha=0.5)
    return ax
```

MDS

4. Considerar la base `df_mds_B.csv`

a. Proyectar las distancias de Manhattan sobre un número adecuado de componentes.

```
# Criterios de evaluación de autovalores
def cSoloPos(eigs):
    sEig = np.sort(eigs)[::-1]
    pos_mask = sEig > 0
    return np.cumsum(sEig) / np.sum(sEig[pos_mask])

def cAbs(eigs):
    sEig = np.sort(eigs)[::-1]
    return np.cumsum(np.abs(sEig)) / np.sum(np.abs(sEig))

def cSq(eigs):
    sEig = np.sort(eigs)[::-1]
    return np.cumsum(sEig**2) / np.sum(sEig**2)

def mds_Mink(df, k, p=1):
    dMat = squareform(pdist(df.____, metric=____, p=p))
    n = dMat.____[____]

    H = np.____(n) - np.____((n, n)) / n
    B = -1/2 * np.dot(np.dot(____, (____**2)), ____)

    eig_vals, eig_vecs = np.linalg.____(____)

    print("Autovalores:")
    print(eig_vals[____])
    print("Criterio Sólo Positivos:")
    print(cSoloPos(eig_vals)[____])
    print("Criterio Módulo:")
    print(cAbs(eig_vals)[____])
    print("Criterio Cuadrados:")
    print(cSq(eig_vals)[____])

    points = np.dot(____[____],np.diag(np.sqrt(____[____])))
    df_mds = pd.DataFrame(points, columns=[f"Dim{i+1}" for i in range(k)], index=df.index)
    return pd.concat([df_mds, df], axis=1)
```

b. Graficar las observaciones sobre las componentes elegidas. Agregar color según otras variables para identificar las variables que guían la disimilitud.

```
def repGrafMDS(df_mds, dims=[1, 2], col_ch=None):
    fig, ax = plt.subplots()
    d1, d2 = f"Dim{dims[0]}", f"Dim{dims[1]}"

    ax.scatter(df_mds[d1], df_mds[d2], alpha=0)
    if col_ch is None:
      for i in range(df_mds.shape[0]):
        ax.text(df_mds.loc[df_mds.____[i],____],df_mds.loc[df_mds.____[i],____],s=df_mds.____[i],alpha=
    else:
      if col_ch in df_mds.columns:
        v_min, v_max = df_mds[____].min(), df_mds[____].max()
```

```
        norm = mcolors.Normalize(vmin=v_min, vmax=v_max)
        cmap = plt.get_cmap("viridis")
        for i in range(df_mds.shape[0]):
          color_texto = cmap(norm(df_mds.loc[df_mds.____[i],____]))
          ax.text(df_mds.loc[df_mds.____[i],____],df_mds.loc[df_mds.____[i],____],s=df_mds.____[i],alpha
        sm = plt.cm.ScalarMappable(cmap="viridis", norm=norm)
        sm.set_array([])
        fig.colorbar(sm, ax=ax, label=____)
      else:
        raise ValueError("Opción de color inválida")

    ax.set_xlabel(d1)
    ax.set_ylabel(d2)
    ax.set_title("Representación MDS")
    ax.grid(True, linestyle='--', alpha=0.5)
    return ax
```